Nicolae Mihai Ghise

Homework 4

Task 1: Understanding the data

For this task I read the drug_screening.csv file into my python script and I displayed the sample distribution of each drug in a bar plot, each drug having between 961-963 cell lines, as Figure 1 shows.

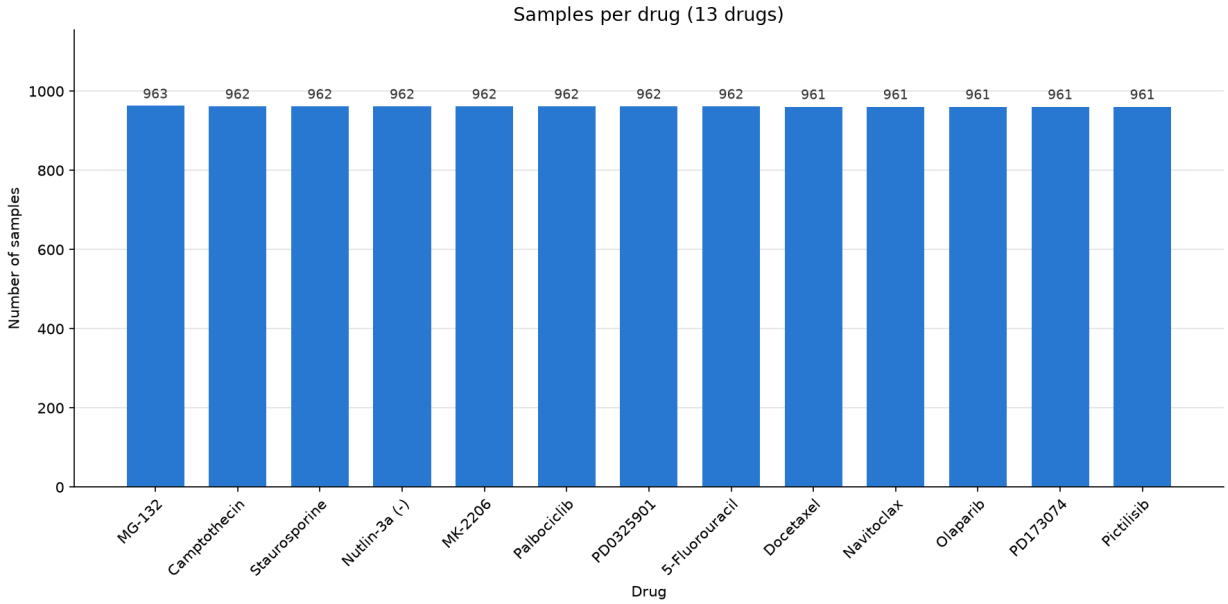


Figure 1. Distribution of drug samples

Task 2: Drug-specific regression analysis

In this task I performed regression analysis for each of the 13 drugs. For this, I used six tree based regression models:

- Decision Trees
- Random Forests
- Gradient Boosting Machines (GBM)
- XGBoost
- LightGBM
- CatBoost

For each model I used 5-fold cross validation in order to split the data into 80% training and 20% testing, shuffling the samples so that each of them ends up at least on time in testing as well.

How each tree regression model works:

1. Decision Tree Regressor: predicts a continuous number by repeatedly splitting the data into smaller groups based on feature values, then predicts the average target value within each final group. It splits at each node, trying every feature and a threshold and picks the split that reduces the error in target the most. It does this recursively, ultimately building a tree of if/else rules. It stops when it hits a limit given by the *max_depth* or *min_samples_leaf* or when the learning stops improving. Then a new sample follows the rules down to a leaf and the prediction is the mean of the training targets in that leaf.
2. Random Forests Regressor: predicts a continuous number by training many decision trees on slightly different versions of the data and then averaging their predictions.

   It bootstraps the samples (*bagging*), each tree being trained on a random sample of the training data. In each bootstrap sample can be repeated as they are chosen randomly. About ⅓ of samples are left out by default, these are the *out-of-bag*s samples.

   At each split the tree considers only a random subset of features given by *max_features* hyperparameter.

   Each new sample goes through every tree and the final prediction is the mean of all the trees' outputs.
3. Gradient Boosting Machines Regressor: predicts a continuous number by building trees one after another, each new tree correcting the errors the previous trees have made so far.

   The start is usually simple, the mean of the target. For each training point, it measures how wrong the current ensemble is by computing the residuals. Fit a small tree to the residuals which learns to predict those errors. The tree's predictions are scaled by a *learning rate* hyperparameter and added to the ensemble. These are repeated for *n_estimators* (hyperparameter) steps, and the final prediction is the initial constant plus the sum of all the scaled tree outputs.
4. XGBoost Regressor: is a gradient boosting machine that adds regularization, optimization on top of the previously described GBM.

   It has the same core principles as the Gradient Boosting Machines, it computes the gradients (first derivative) of the loss, but also the *Hessians* (second derivative) of the loss, which gives more accurate tree fits than only the gradient does. Regularization plays a role in minimizing the loss by adding a complexity penalty, which grows with the number of leaves.
5. LightGBM Regressor: is a gradient boosting implementation built for speed and memory efficiency, especially for larger datasets.

   It basically has the same principals as the last two models described, but it uses *histogram binning*, meaning that features are bucketed into bins before training, which makes split search faster and uses less memory.

   It applies leaf-wise tree growth: instead of growing a full level at a time, it always splits the single leaf with the largest loss reduction, this gives deeper and asymmetric trees that reduce the loss function faster.

6. CatBoost Regressor: it's also a gradient boosting machine implementation that handles categorical features well and reduces overfitting.

   Same principles as GBM, but it keeps track of the target statistics for categories. It's symmetric, every node at the same depth uses the same split condition, trees being balanced and simple which makes prediction fast.

After training each of these tree based models, I evaluated them on the testing dataset and saved the results together in order to choose the best regressor for each drug. The evaluation was done using the following 4 regression metrics:

- Mean Absolute Error: $MAE = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_i\right|$
- Mean Squared Error: $MSE = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2$
- Root Mean Squared Error: $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2}$
- Coefficient of Determination: $R^2 = 1 - \frac{\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2}{\sum_{i=1}^{n}\left(y_i - \bar{y}\right)^2} = 1 - \frac{SS_{res}}{SS_{tot}}$

In order to choose the best regressor for each drug we have to look for the minimum MAE, MSE and RMSE, and the highest R^2. Below are the bar plots of the evaluation metrics for each of the 13 drugs given, the best model has its metric bounded in a rectangle.

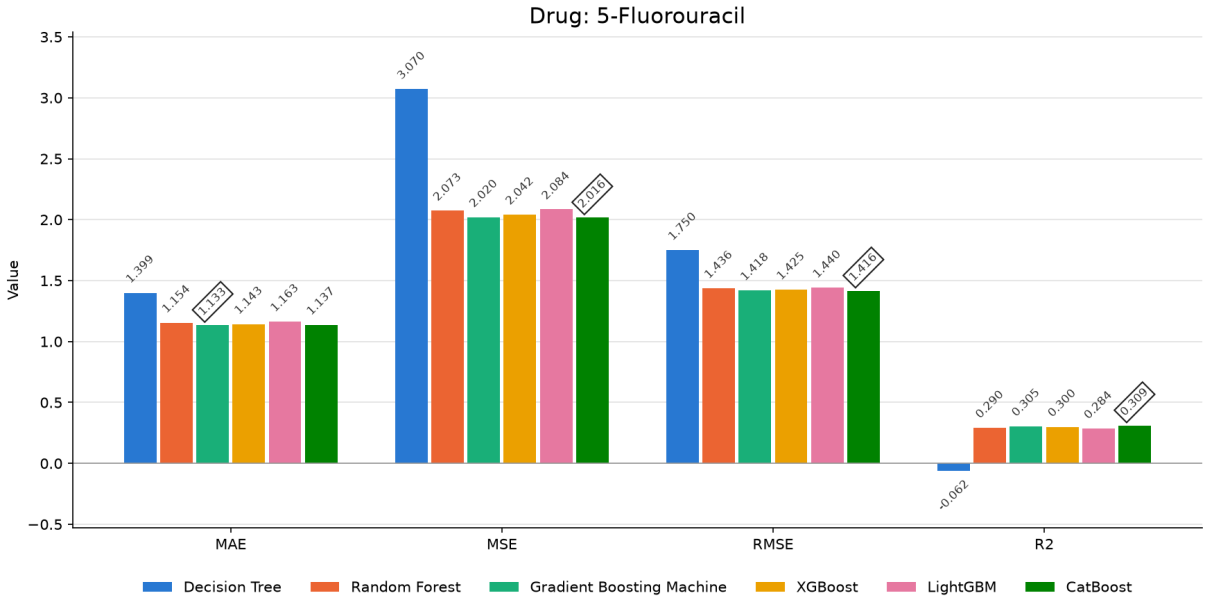


Figure 2. 5-Fluorouracil Evaluation

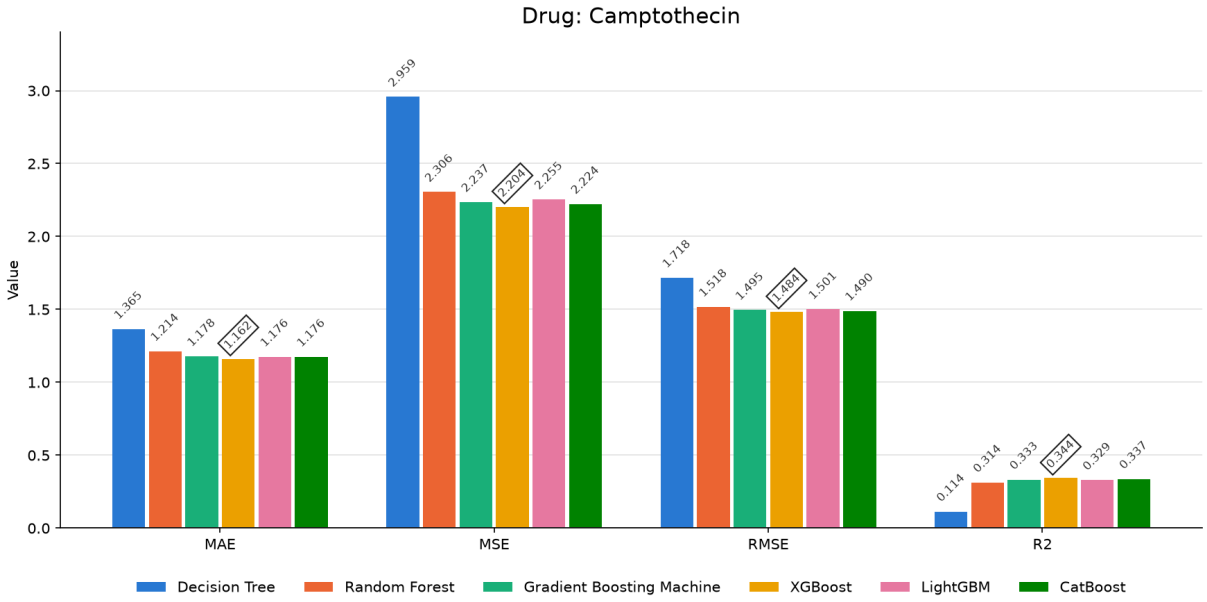


Figure 3. Camptothecin Evaluation

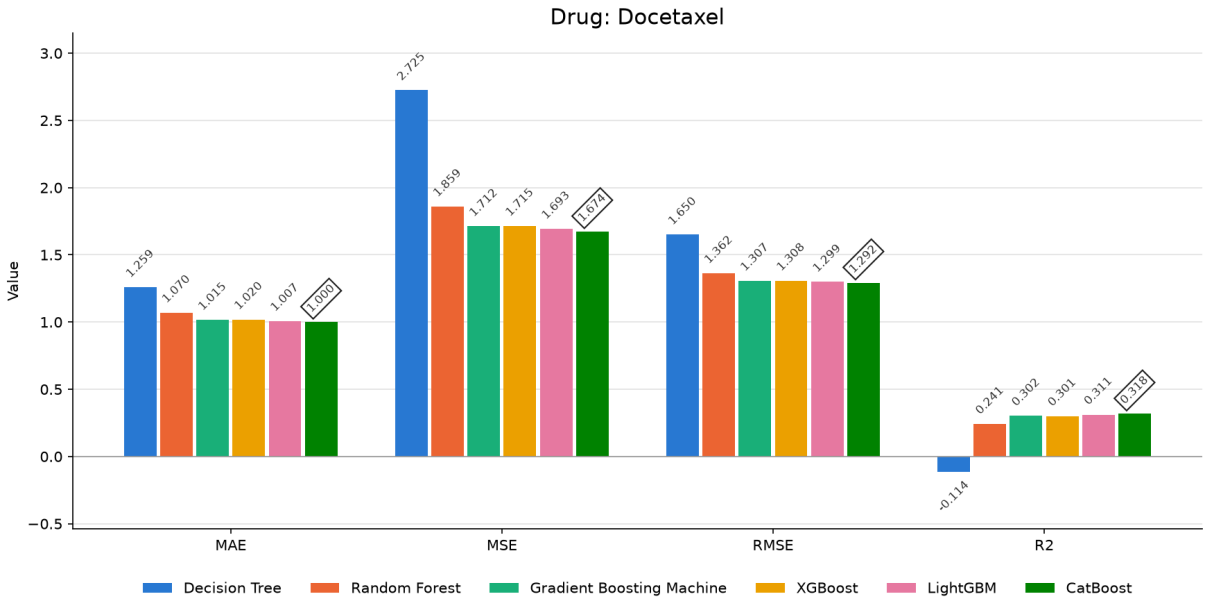


Figure 4. Docetaxel Evaluation

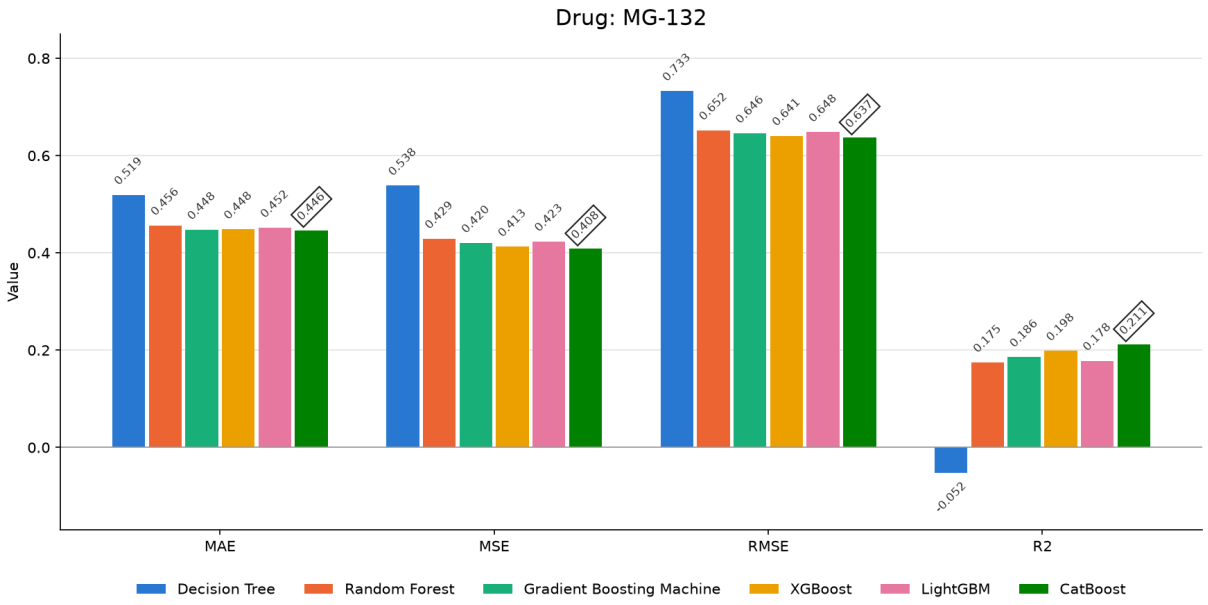


Figure 5. MG-132 Evaluation

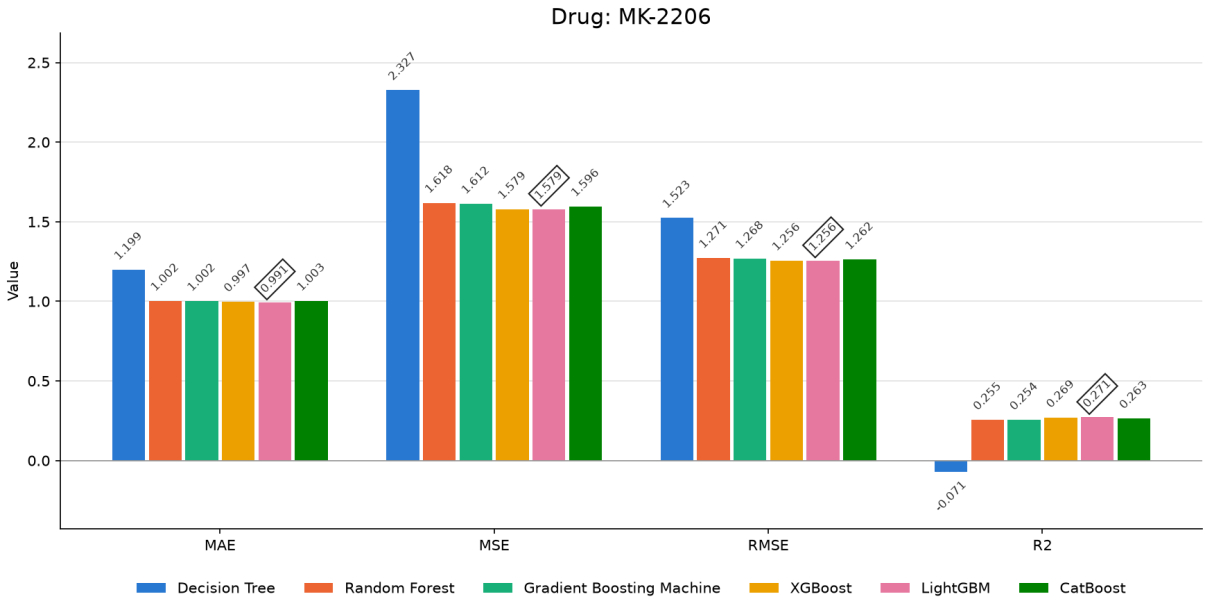


Figure 6. MK-2206 Evaluation

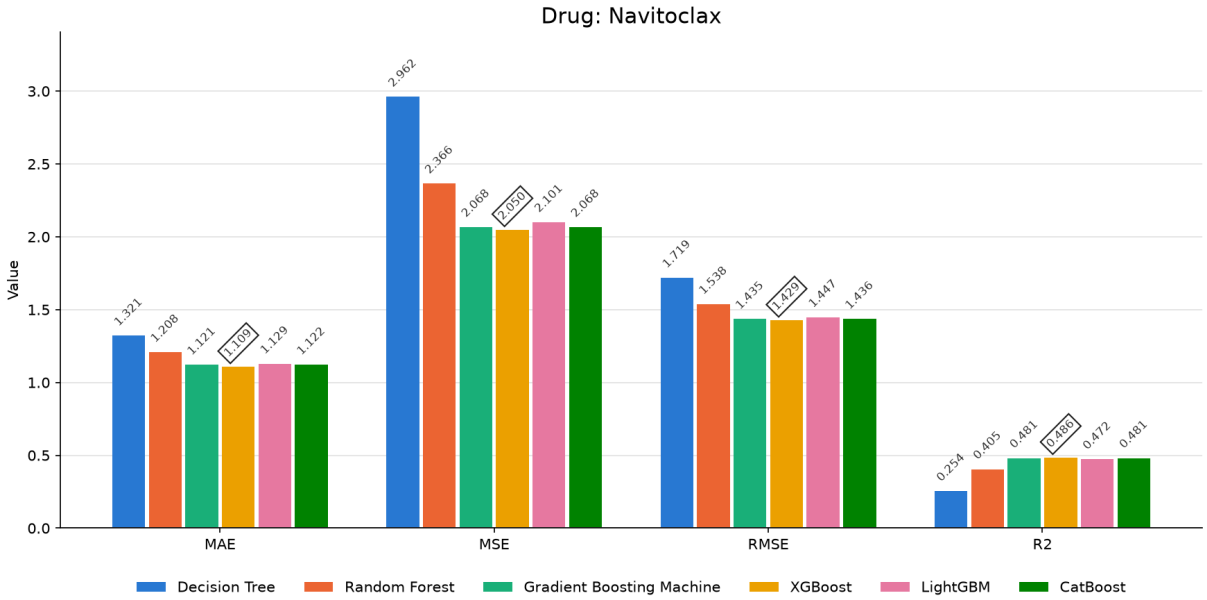


Figure 7. Navitoclax Evaluation

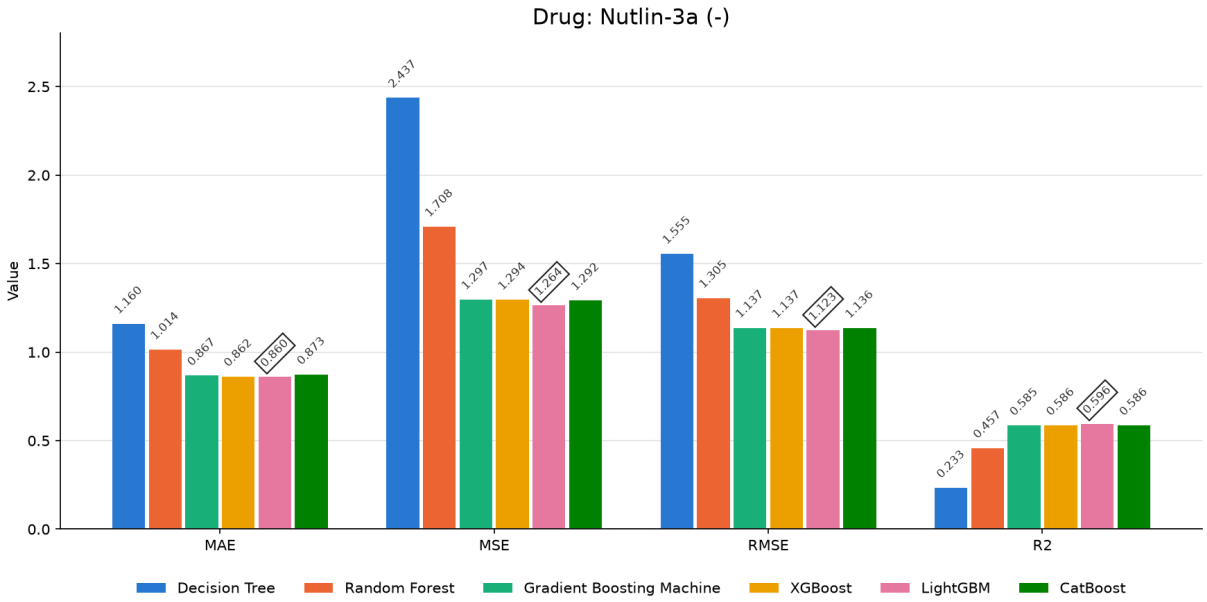


Figure 8. Nutlin-3a Evaluation

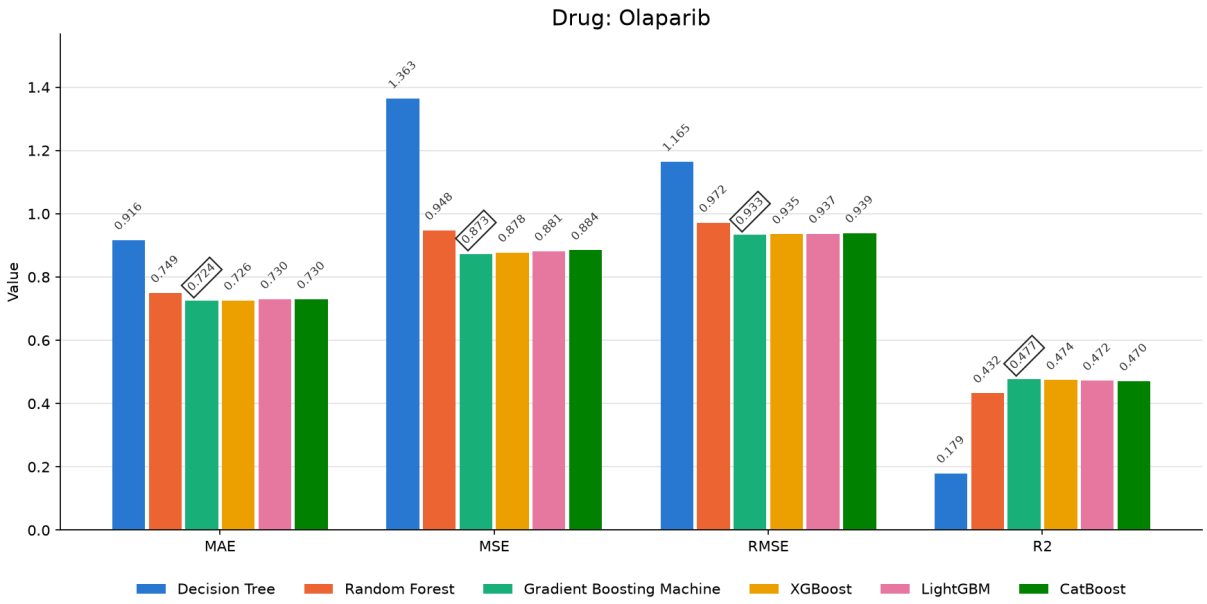


Figure 9. Olaparib Evaluation

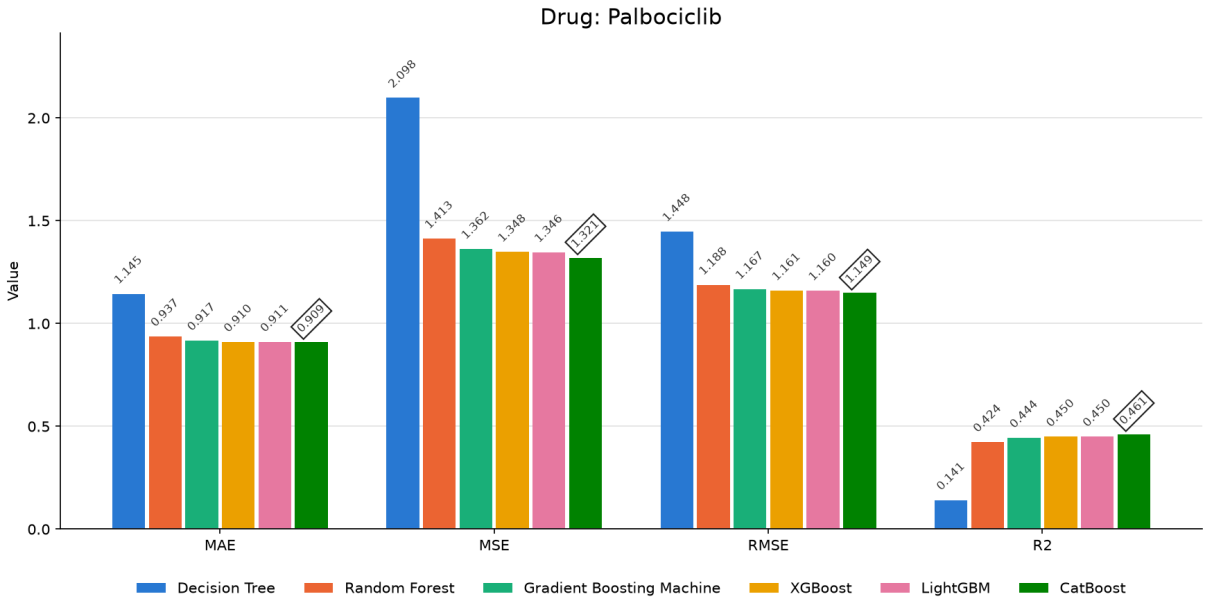


Figure 10. Palbociclib Evaluation

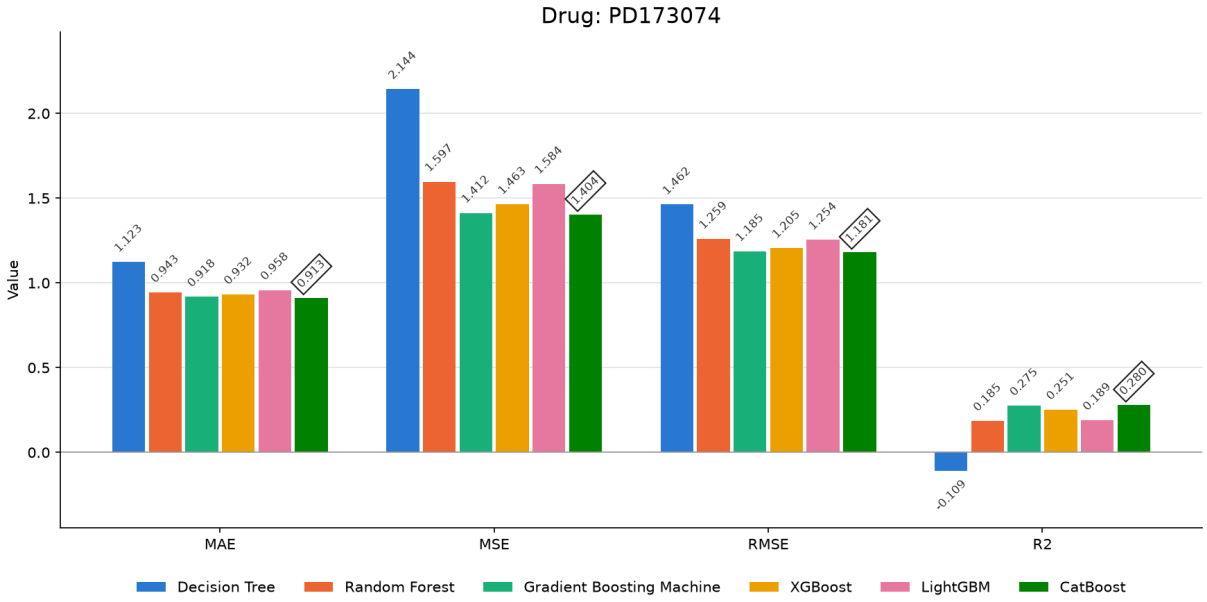


Figure 11. PD173074 Evaluation

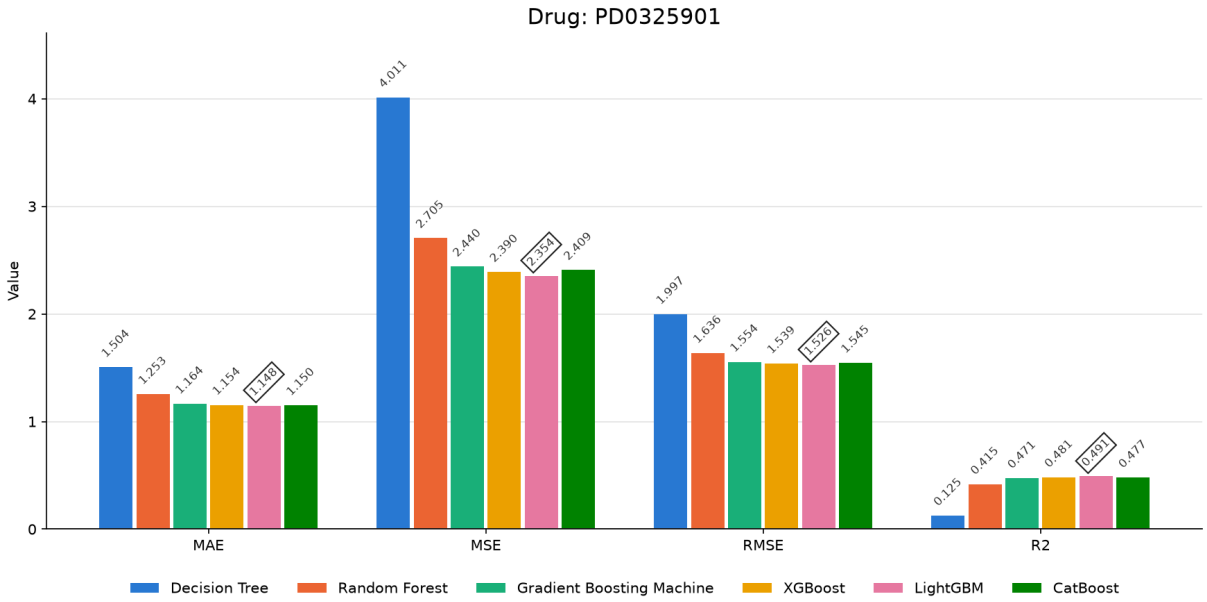


Figure 12. PD0325901 Evaluation

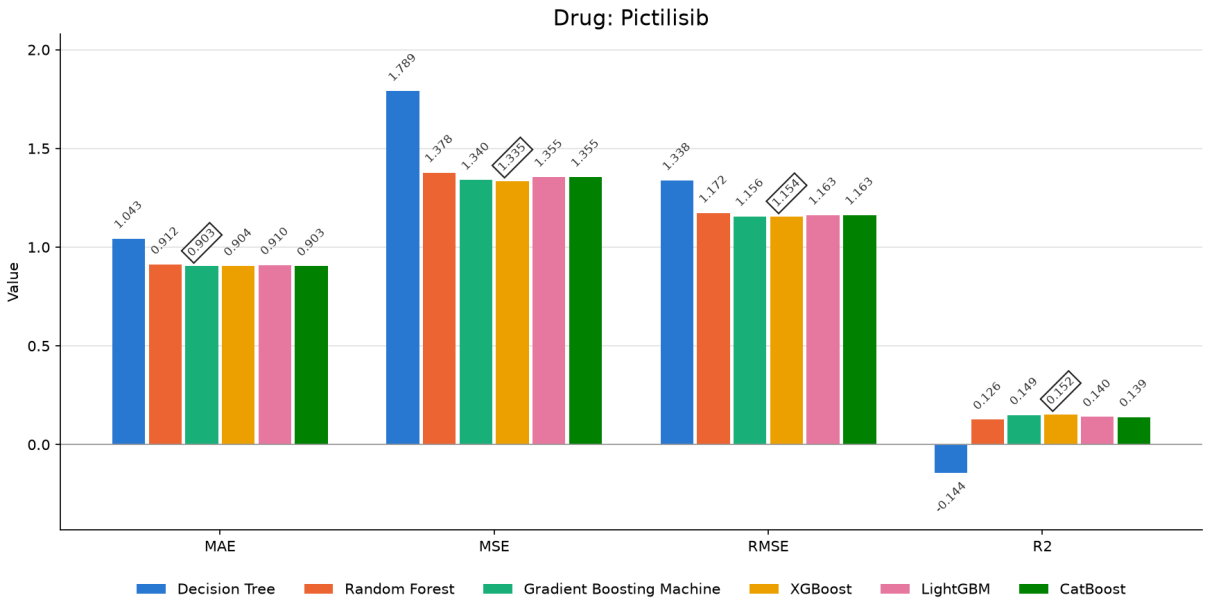


Figure 13. Pictilisib Evaluation

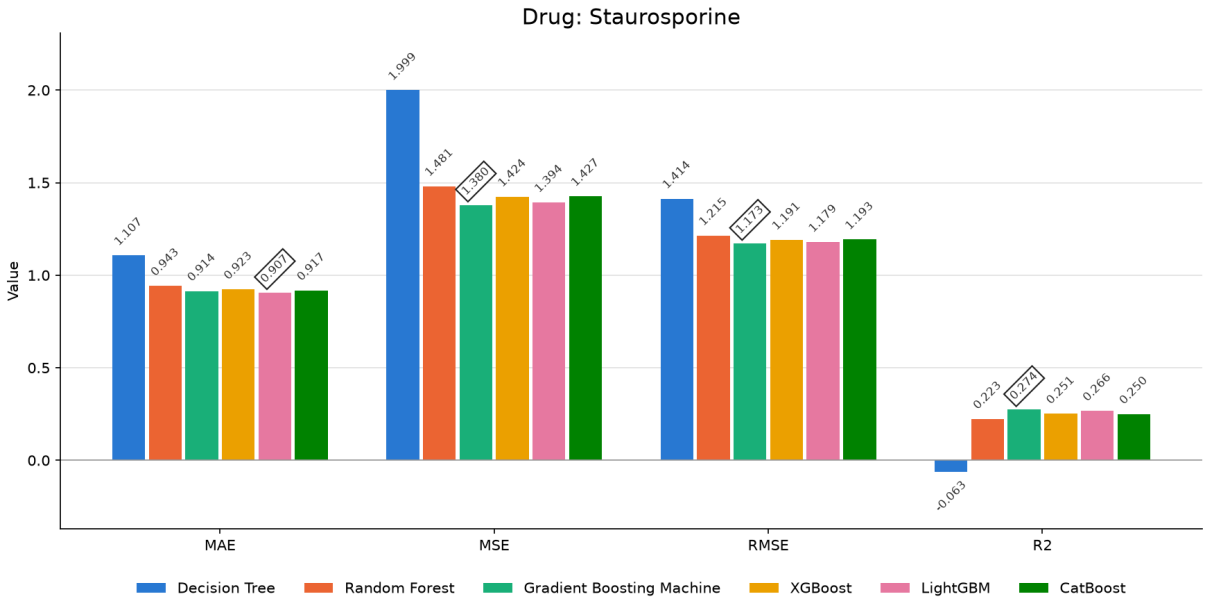


Figure 14. Staurosporine Evaluation

From these plots we can extract the best model for each drug by looking at the bordered scores. In my example I'll be using the lowest RMSE, but anybody can use either the lowest MAE, MSE or the highest R^2.

Task 3:

For this task, for each of the 13 drugs I applied SHAP on the best regressor model. The SHAP algorithm will identify and illustrate the top 10 genes that mostly contribute to the regression predictions (either positively or negatively). Below are the 13 SHAP plots illustrating the 10 most important feature genes for each drug, applied on the best regressor for that specific drug.

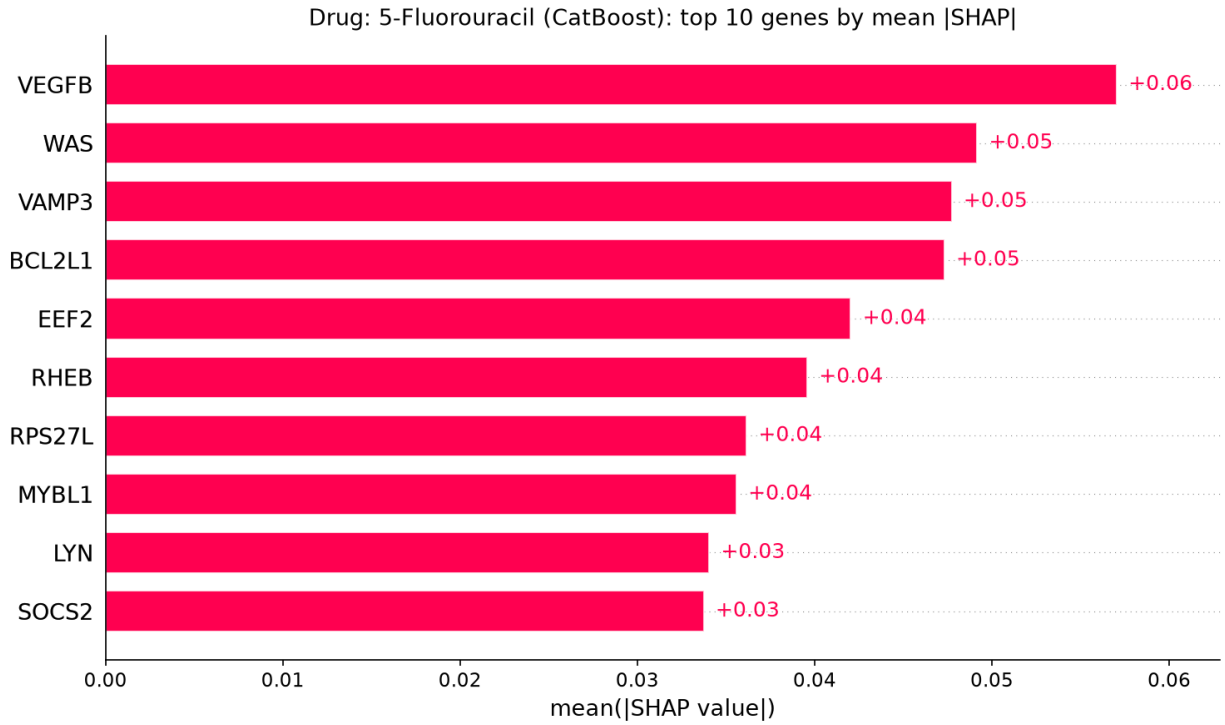


Figure 15. Top 10 genes for 5-Fluorouracil on CatBoost Regressor

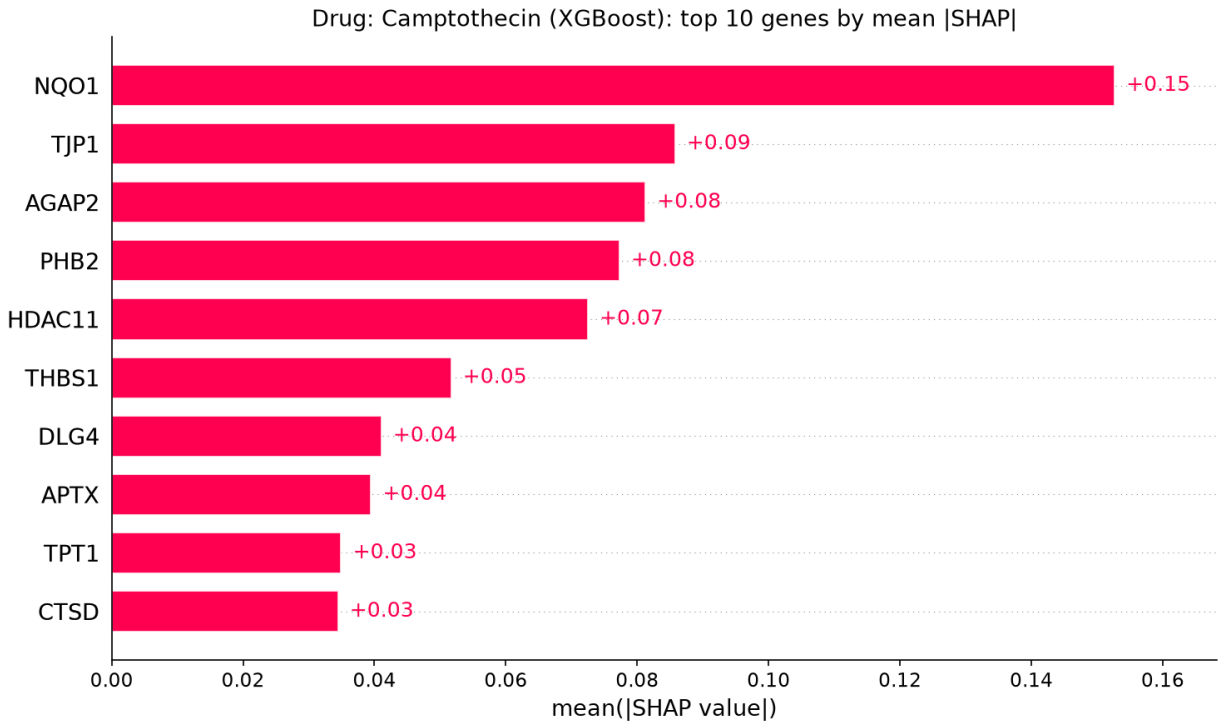


Figure 16. Top 10 genes for Camptothecin on XGBoost Regressor

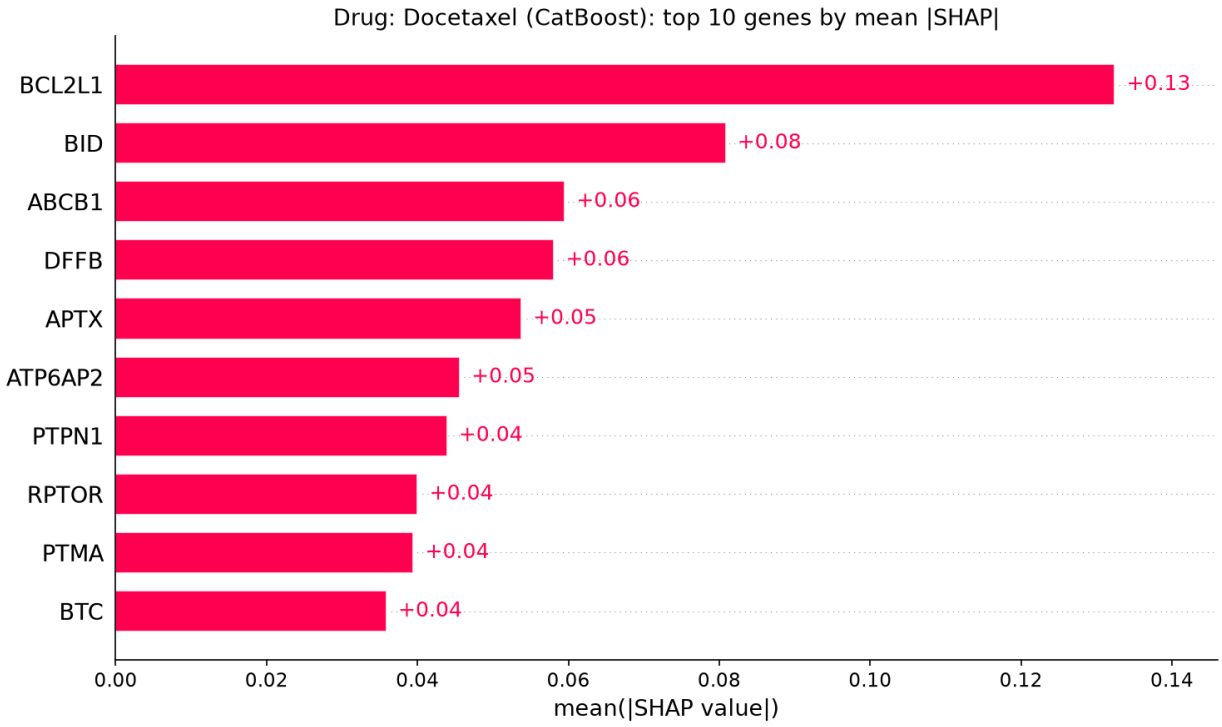


Figure 17. Top 10 genes for Docetaxel on CatBoost Regressor

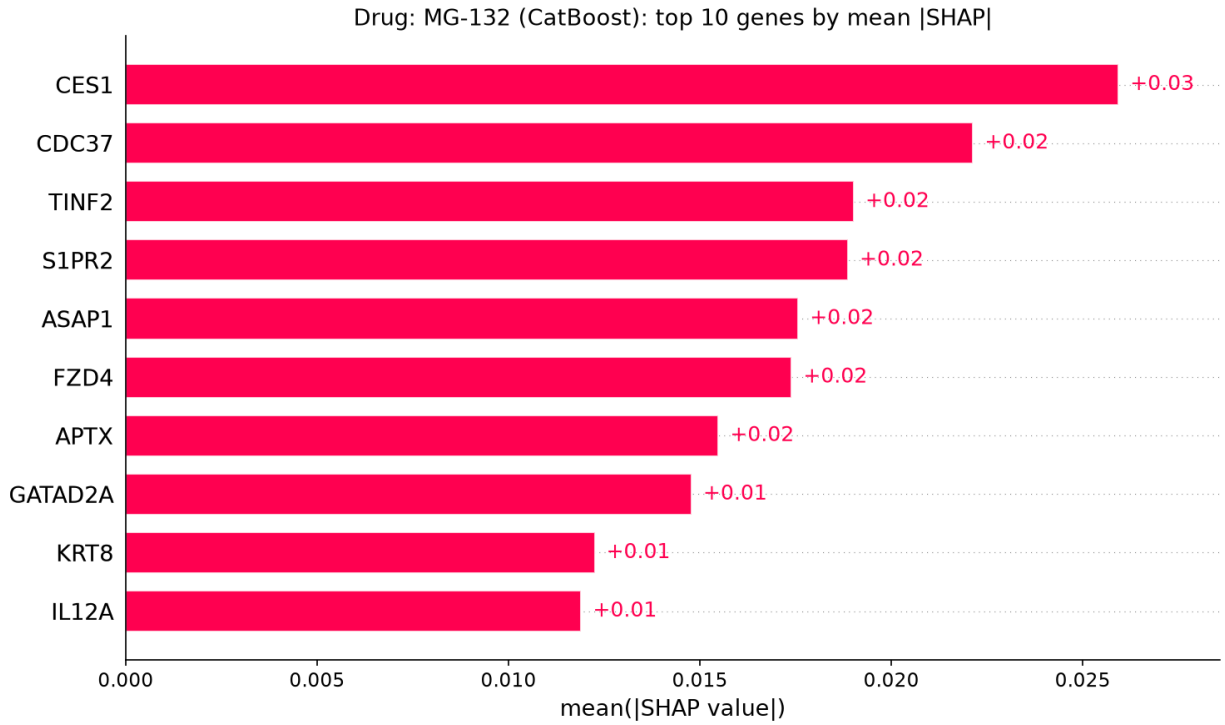


Figure 18. Top 10 genes for MG-132 on CatBoost Regressor

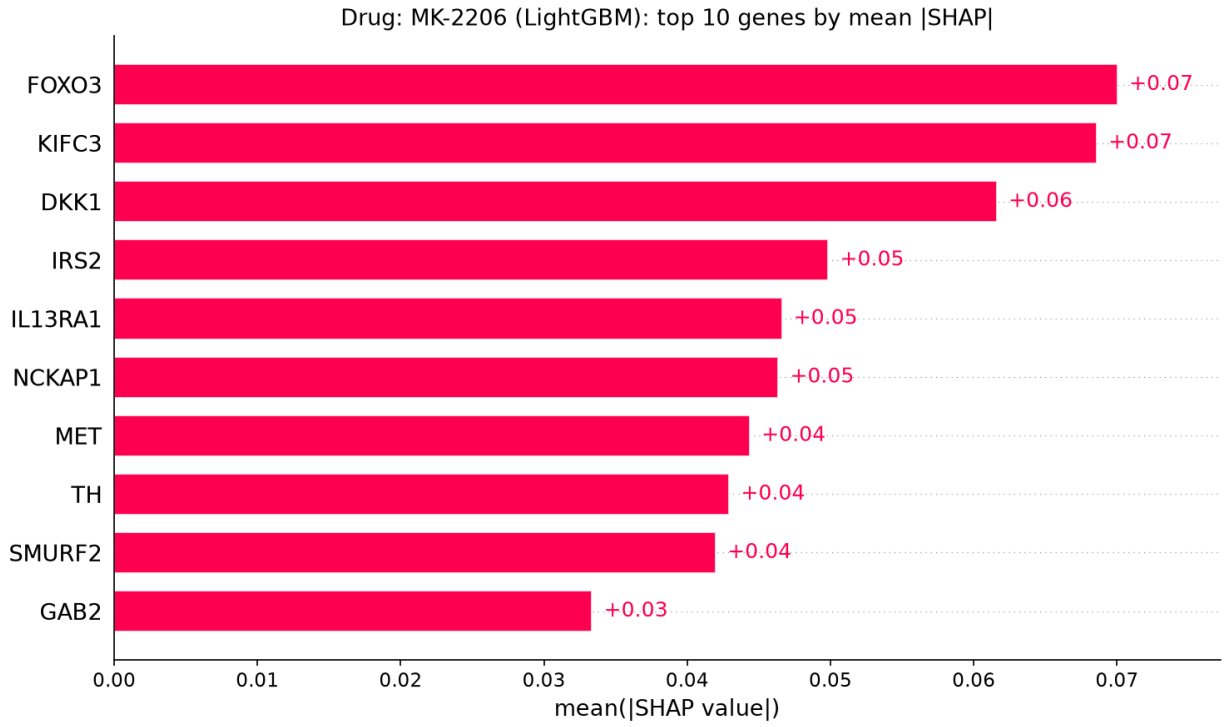


Figure 19. Top 10 genes for MK-2206 on LightGBM Regressor

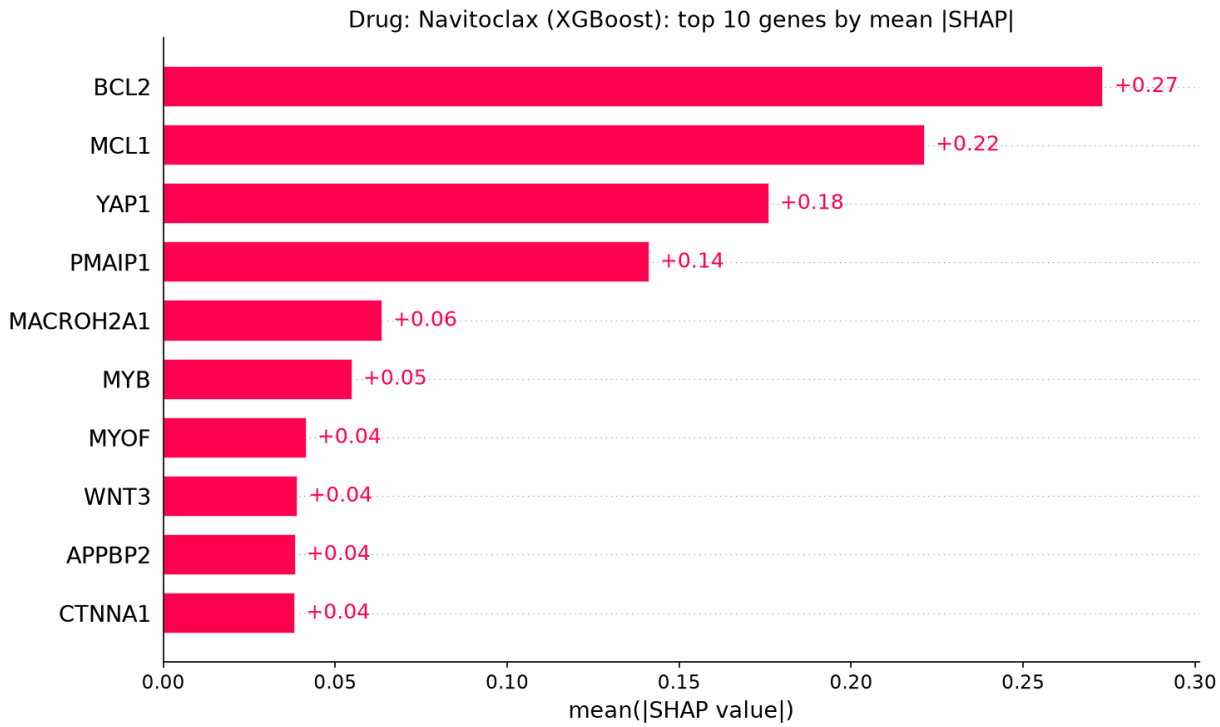


Figure 20. Top 10 genes for Navitoclax on XGBoost Regressor

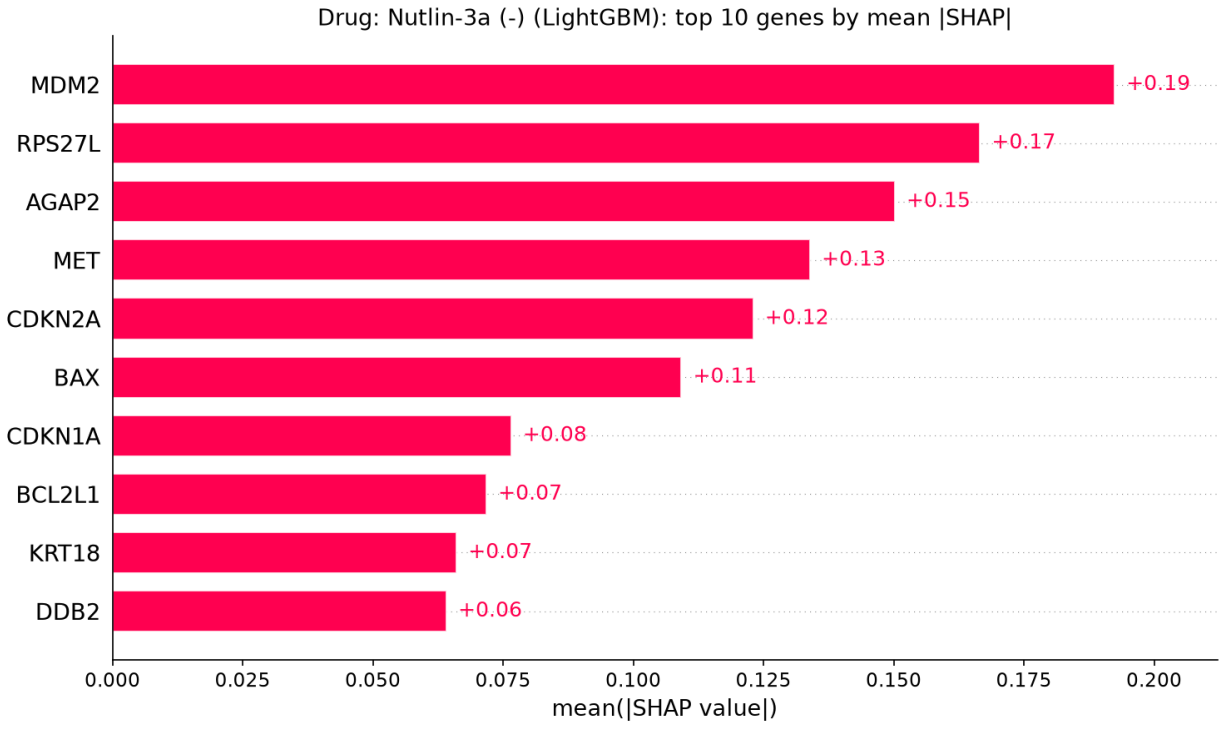


Figure 21. Top 10 genes for Nutlin-3a on LightGBM Regressor

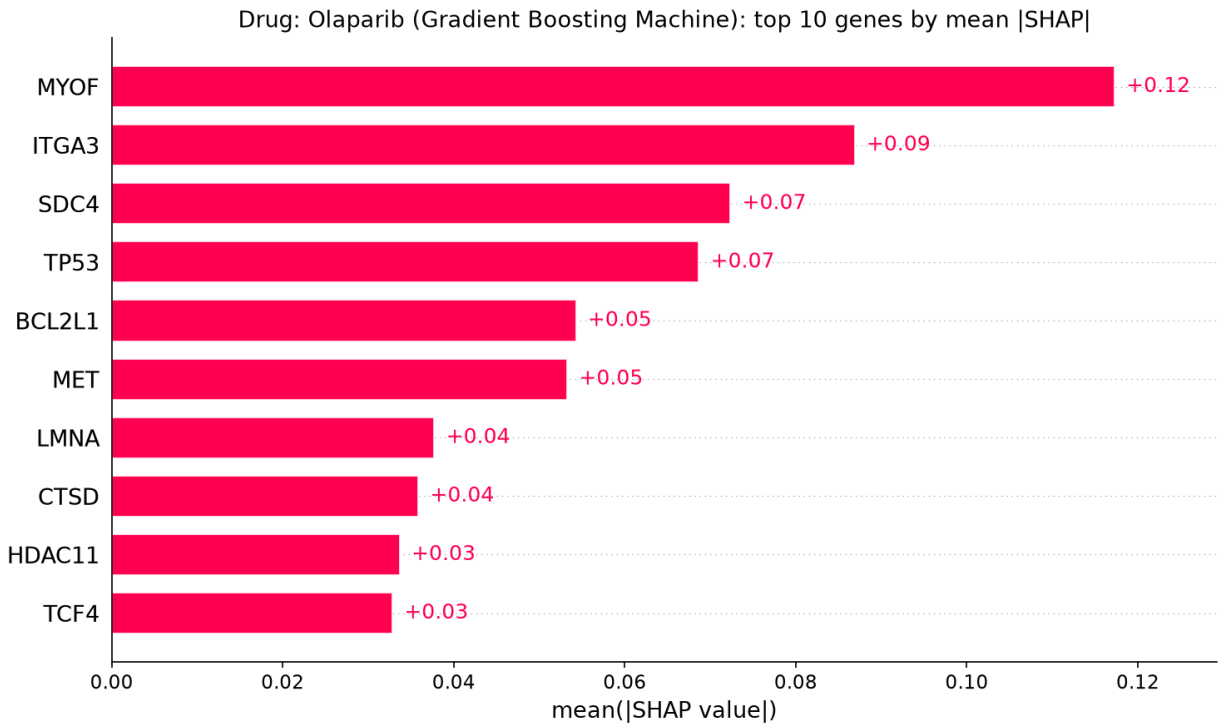


Figure 22. Top 10 genes for Olaparib on GBM Regressor

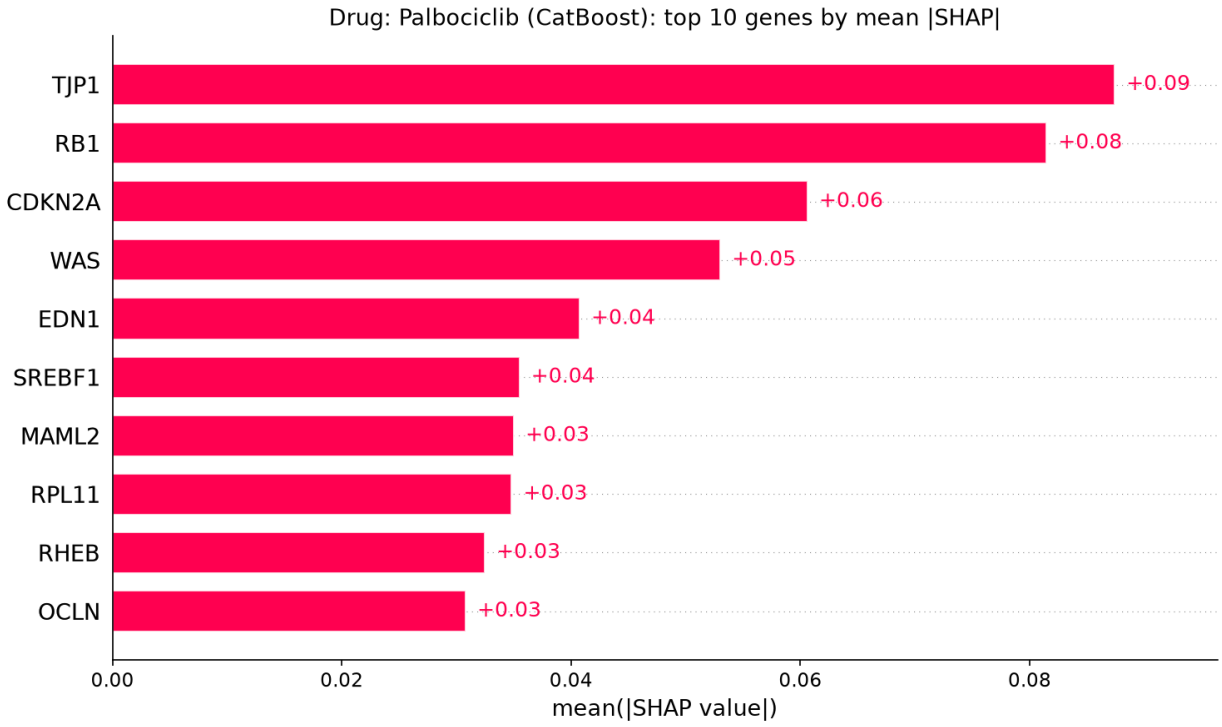


Figure 23. Top 10 genes for Palbociclib on CatBoost Regressor

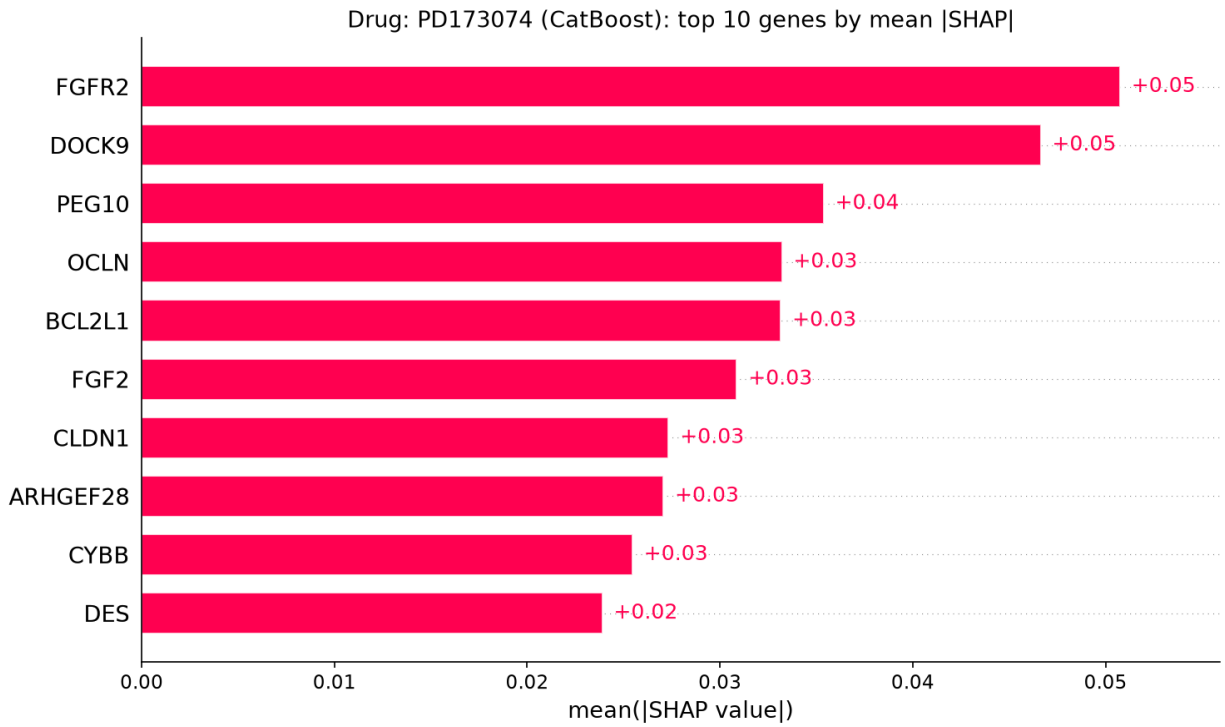


Figure 24. Top 10 genes for PD173074 on CatBoost Regressor

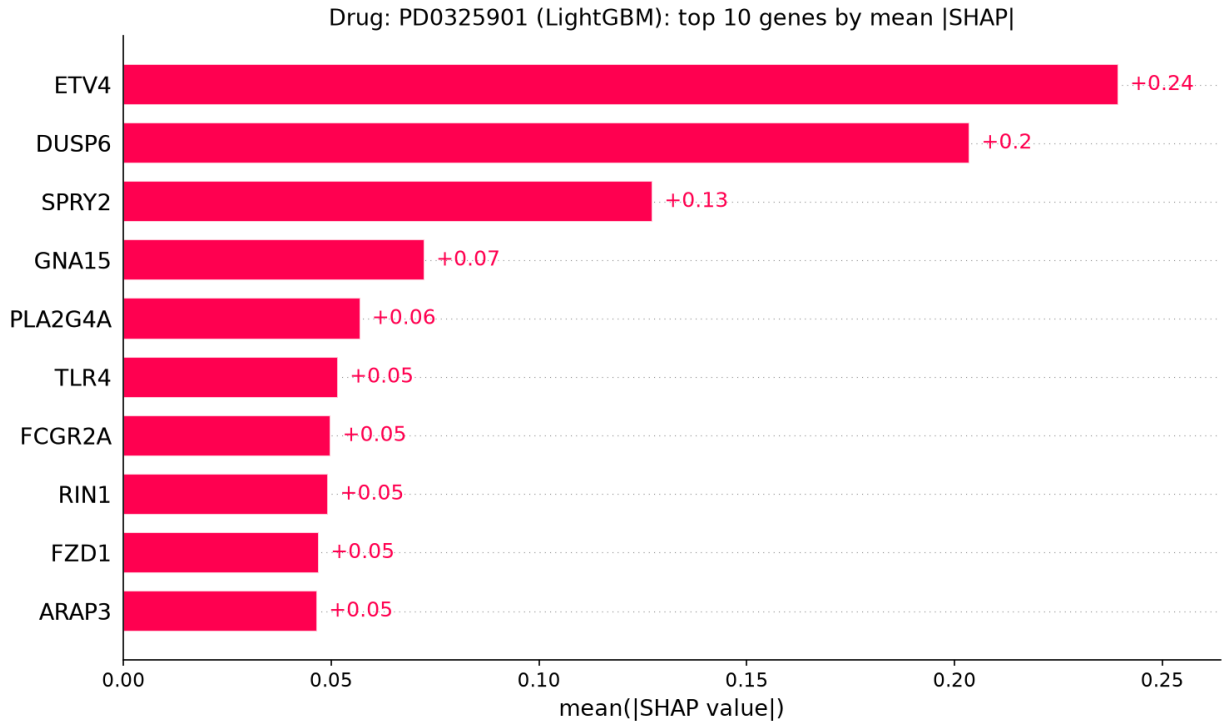


Figure 25. Top 10 genes for PD0325901 on LightGBM Regressor

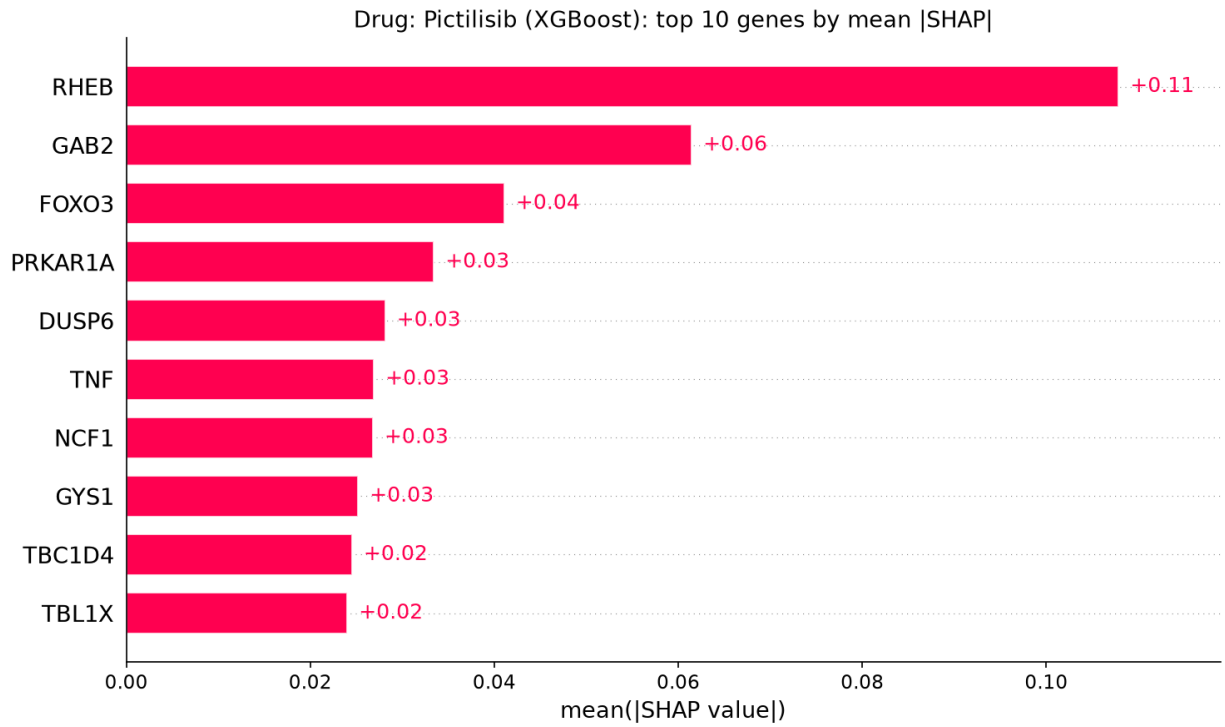


Figure 26. Top 10 genes for Pictilisib on XGBoost Regressor

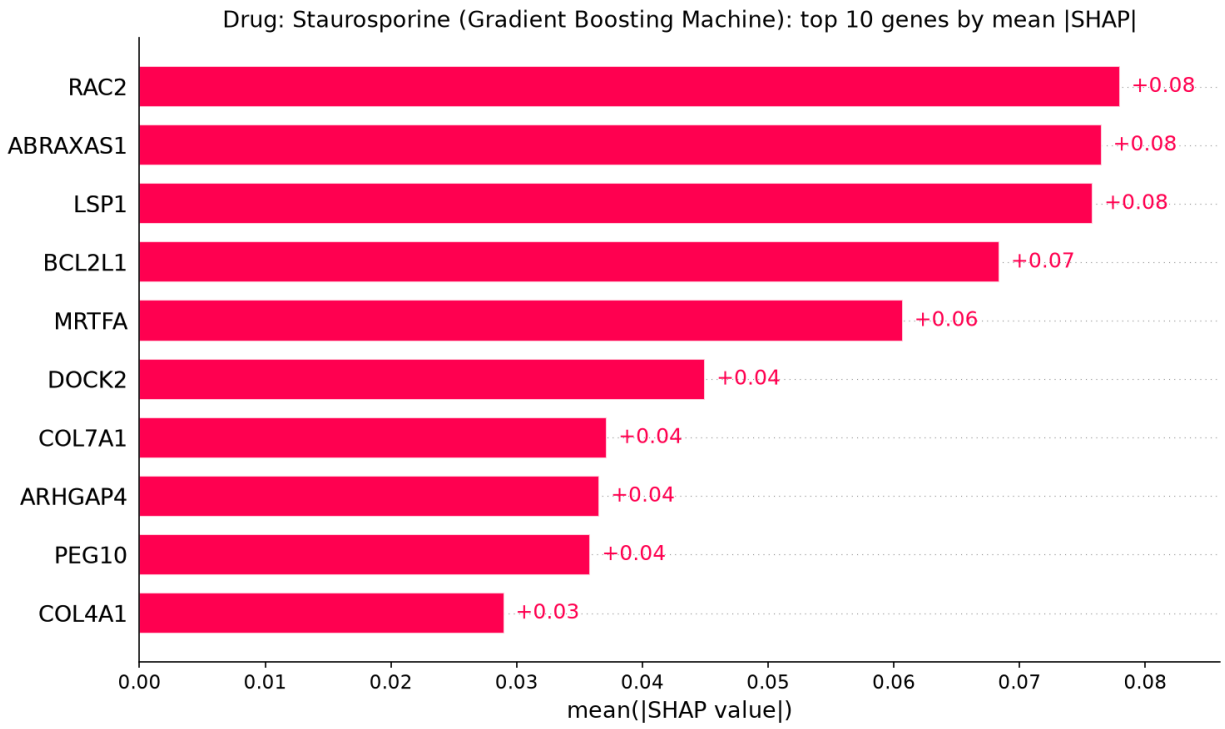


Figure 27. Top 10 genes for Staurosporine on GBM Regressor

Observations:

In Figure 20, for the drug *Navitoclax*, using the SHAP on XGBoost, it can be seen there is a gene that dominates the score prediction: *BCL2* with an absolute value of *0.27* and then, right behind it is *MCL1* with absolute value of *0.22* . This shows that there are some genes for *Navitoclax* drugs that contribute significantly to the prediction score.

On the other hand, if we look at Figure 24, for drug *PD173074* on CatBoost Regressor we see that even the top 10 genes have a value equal or lower to *0.05*, meaning that almost no feature gene in the dataset has a significant contribution to the prediction score of the regression model.

It seems that CatBoost Regressor performed better on most of the drugs.

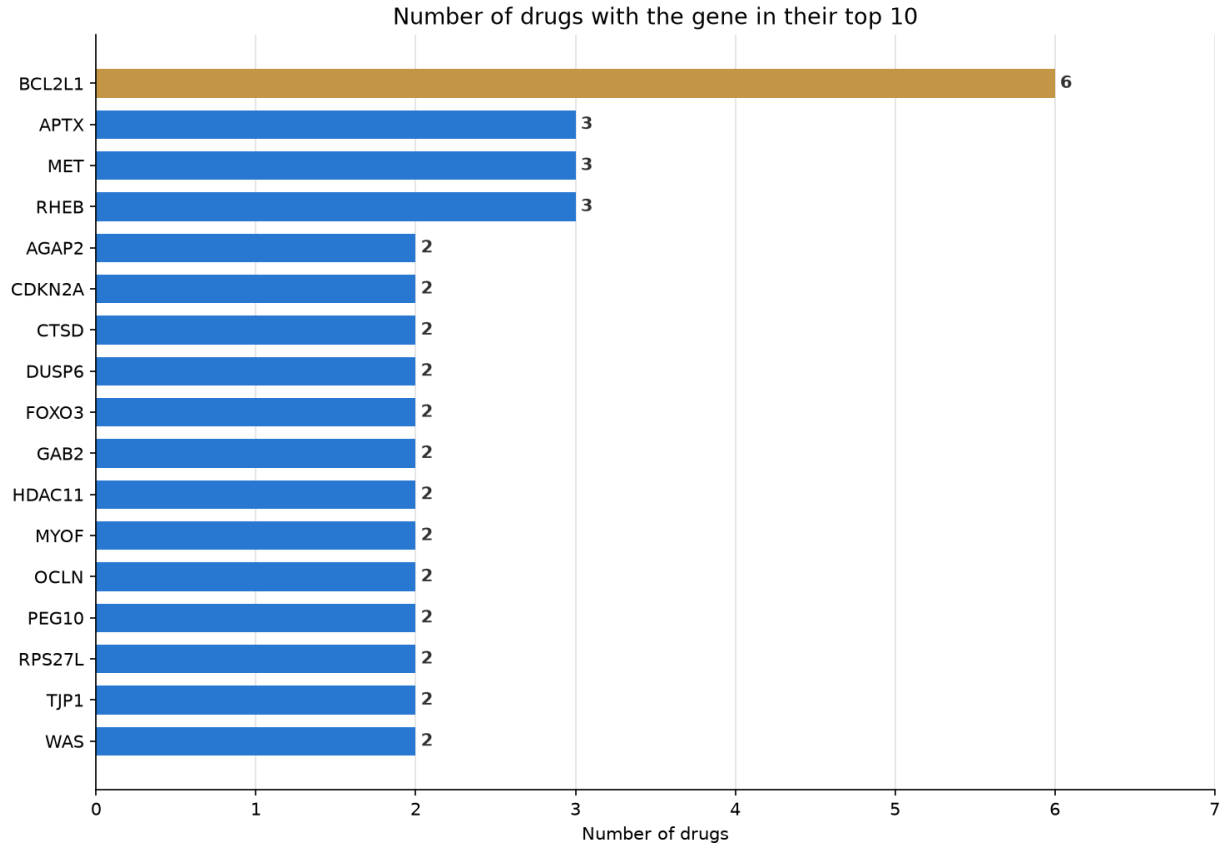


Figure 28. Genes with the highest number of appearances in top 10 for each drug

Above in Figure 28 it can be seen that the feature gene *BCL2L1* appears in the top 10 genes that influence a drug the most in 6 different drugs out of the 13, about half. It is followed by *APTX*, *MET* and *RHEB*, which appear in 3 of the 13 drugs.